

LABORATORY ASSIGNMENT

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 2

Student Name: K.Vinay

Roll No.: A24126552086

Date: 04-10-2026

1. TITLE

Student Database Management System using MongoDB and Node.js

2. AIM

To design and implement a MongoDB database system for managing student records, demonstrating fundamental CRUD operations and querying mechanisms using both the MongoDB Shell and the Node.js MongoDB driver.

3. OBJECTIVE

The objectives of this assignment are:

- To establish a connection to a local MongoDB instance.
- To create a database and a collection for student records.
- To insert multiple student documents into the collection.
- To perform advanced search queries using operators like \$gt, \$lt, and \$set.
- To update and delete specific student documents.
- To sort documents and implement indexes for query performance enhancement.

4. THEORY

MongoDB is a NoSQL, document-oriented database that stores data in flexible, JSON-like BSON documents. It does not require a rigid schema, making it ideal for evolving applications. In MongoDB, data is organized into Databases and Collections (equivalent to tables in SQL).

CRUD Operations: Create (insert), Read (find), Update (updateOne), Delete (deleteOne).

Query Operators: Operators like \$gt (greater than), \$lt (less than), and \$set (modify specific field) allow powerful data filtering and mutations.

Indexes: Creating an index on highly queried fields (like rollNo) transforms operations from linear Collection Scans (COLLSCAN) to efficient Index Scans (IXSCAN).

5. ALGORITHM / PROCEDURE

1. Connect to MongoDB locally on port 27017 and initialize 'collegeDB' database and 'students' collection.
2. Insert 5-8 JSON objects containing rollNo, name, branch, year, marks, and email.
3. Read and display all student records, and then selectively display students based on branch or marks conditions.
4. Locate specific students by Roll No and update their marks and email fields.

5. Delete a specific student using their Roll No.
6. Sort the entire collection of students in descending order based on marks.
7. Create a unique index on the rollNo field and execute .explain('executionStats') to prove index utilization.

6. PROGRAM

student_management.js

```
const { MongoClient } = require('mongodb');

// Connection URL and Database
const url = 'mongodb://127.0.0.1:27017';
const client = new MongoClient(url);
const dbName = 'collegeDB';

async function main() {
  try {
    await client.connect();
    console.log('Connected to MongoDB successfully.');
```

```

// 5. Search for a student using rollNo
console.log('\n--- Search Student by rollNo (23CM001) ---');
console.log(await students.findOne({ rollNo: "23CM001" }));

// 6. Search students based on condition: year = 3 and marks >= 80
console.log('\n--- Year 3 Students with Marks >= 80 ---');
console.log(await students.find({ year: 3, marks: { $gte: 80 } }).toArray());

// 7. Update marks of a particular student
await students.updateOne({ rollNo: "23CM001" }, { $set: { marks: 92 } });
console.log('\nUpdated marks for 23CM001 to 92');

// 8. Update another field (email)
await students.updateOne({ rollNo: "23CM003" }, { $set: { email:
"ananya.official@example.com" } });
console.log('Updated email for 23CM003');

// 9. Delete a student record using rollNo
await students.deleteOne({ rollNo: "23CM005" });
console.log('\nDeleted student record for 23CM005');

// 10. Display students in descending order of marks
console.log('\n--- Students Sorted by Marks (Descending) ---');
console.log(await students.find().sort({ marks: -1 }).toArray());

// 11. Create an index on rollNo
await students.createIndex({ rollNo: 1 }, { unique: true });
console.log('\nCreated unique index on rollNo');

// 12. Demonstrate index performance (explain executionStats)
const stats = await students.find({ rollNo: "23CM007" }).explain("executionStats");
console.log('\nIndex Search Stage:', stats.queryPlanner.winningPlan.stage);

// 13. Real-Time Extension Queries
console.log('\n--- Real-Time Extension Queries ---');
console.log('Marks > 80:', await students.find({ marks: { $gt: 80 } }).toArray());
console.log('Marks < 50:', await students.find({ marks: { $lt: 50 } }).toArray());
console.log('Highest Scorer:', await students.find().sort({ marks: -1
}).limit(1).toArray());
console.log('IT Branch:', await students.find({ branch: "IT" }).toArray());
console.log('Sorted Ascending:', await students.find().sort({ marks: 1 }).toArray());

} catch (err) {
  console.error('Database Error:', err.message);
} finally {
  await client.close();
  console.log('\nConnection closed.');
```

```

}
main();

```

queries.js

```

// MongoDB Shell Script: queries.js
// Database: collegeDB | Collection: students
// Assignment 2: Student Database Management System

```

```

// 1. Switch to collegeDB database
usecollegeDB;

// 2. Insert initial student records
db.students.insertMany([
  { rollNo: "23CM001", name: "Ravi Kumar", branch: "CSE-AIML", year: 3, marks: 85, email:
"ravi@example.com" },
  { rollNo: "23CM002", name: "Priya Sharma", branch: "CSE", year: 3, marks: 92, email:
"priya@example.com" },
  { rollNo: "23CM003", name: "Ananya Verma", branch: "IT", year: 2, marks: 78, email:
"ananya@example.com" },
  { rollNo: "23CM004", name: "Kiran Patel", branch: "ECE", year: 4, marks: 45, email:
"kiran@example.com" },
  { rollNo: "23CM005", name: "Suresh Reddy", branch: "CSE-AIML", year: 3, marks: 68, email:
"suresh@example.com" },
  { rollNo: "23CM006", name: "Sneha Nair", branch: "CSE", year: 2, marks: 88, email:
"sneha@example.com" },
  { rollNo: "23CM007", name: "Ramesh Rao", branch: "CSE", year: 3, marks: 95, email:
"ramesh@example.com" },
  { rollNo: "23CM008", name: "Vikram Das", branch: "IT", year: 1, marks: 48, email:
"vikram@example.com" }
]);

// 3. Display all students
print("\n--- 1. All Students ---");
db.students.find().pretty();

// 4. Display students belonging to a particular branch
print("\n--- 2. Students in CSE-AIML Branch ---");
db.students.find({ branch: "CSE-AIML" }).pretty();

// 5. Display students who scored more than 75 marks
print("\n--- 3. Students Scoring More than 75 Marks ---");
db.students.find({ marks: { $gt: 75 } }).pretty();

// 6. Search for a student using rollNo
print("\n--- 4. Search Student by rollNo (23CM001) ---");
db.students.findOne({ rollNo: "23CM001" });

// 7. Search students based on condition (year = 3 AND marks >= 80)
print("\n--- 5. Year 3 Students with Marks >= 80 ---");
db.students.find({ year: 3, marks: { $gte: 80 } }).pretty();

// 8. Update marks of a particular student
print("\n--- 6. Update Marks for rollNo 23CM001 to 92 ---");
db.students.updateOne(
  { rollNo: "23CM001" },
  { $set: { marks: 92 } }
);

// 9. Update another field (email)
print("\n--- 7. Update Email for rollNo 23CM003 ---");
db.students.updateOne(
  { rollNo: "23CM003" },
  { $set: { email: "ananya.official@example.com" } }
);

```

```
// 10. Delete a student record using rollNo
print("\n--- 8. Delete Student Record (23CM005) ---");
db.students.deleteOne({ rollNo: "23CM005" });

// 11. Display students in descending order of marks
print("\n--- 9. Students in Descending Order of Marks ---");
db.students.find().sort({ marks: -1 }).pretty();

// 12. Create a unique index on rollNo
print("\n--- 10. Create Unique Index on rollNo ---");
db.students.createIndex({ rollNo: 1 }, { unique: true });

// 13. Demonstrate Index Performance (Execution Statistics)
print("\n--- 11. Index Performance (IXSCAN vs COLLSCAN) ---");
db.students.find({ rollNo: "23CM007" }).explain("executionStats");

// 14. Real-Time Extension Queries ★
print("\n--- Real-Time Extension Queries ---");

// Extension A: Students scoring above 80
print("\n[Extension A] Students Scoring Above 80:");
db.students.find({ marks: { $gt: 80 } }).pretty();

// Extension B: Students scoring below 50
print("\n[Extension B] Students Scoring Below 50:");
db.students.find({ marks: { $lt: 50 } }).pretty();

// Extension C: Highest-scoring student (Topper)
print("\n[Extension C] Highest-Scoring Student:");
db.students.find().sort({ marks: -1 }).limit(1).pretty();

// Extension D: Students belonging to IT branch
print("\n[Extension D] Students in IT Branch:");
db.students.find({ branch: "IT" }).pretty();

// Extension E: Students sorted ascending by marks
print("\n[Extension E] Students Sorted by Marks (Ascending):");
db.students.find().sort({ marks: 1 }).pretty();
```

7. CODE EXPLANATION

Code / Syntax	Explanation
<code>new MongoClient(url)</code>	Instantiates a client connection to the MongoDB server in Node.js.
<code>db.collection('students')</code>	References the target collection.
<code>insertMany([...])</code>	Adds multiple JSON documents simultaneously to the database.
<code>{ marks: { \$gt: 75 } }</code>	Example of a filter object finding students scoring strictly above 75.
<code>updateOne({ rollNo: '...' }, { \$set: { ... } })</code>	Locates a document by ID and updates specific keys without overwriting the whole document.
<code>sort({ marks: -1 })</code>	Sorts the cursor output in descending order (-1) based on the marks key.

createIndex({ rollNo: 1 }, { unique: true })	Ensures the rollNo field is indexed for fast lookup and prevents duplicates.
--	--

8. TEST CASES AND EXECUTION DATA

Test Case	Script / Input	Expected Output	Actual Output	Status
TC-01	Connect and Insert data	Data added to MongoDB	Records successfully inserted	Pass
TC-02	Query by Condition (marks > 75)	Return filtered documents	Correct students returned	Pass
TC-03	Update operations	Update marks & email	Documents reflect changes successfully	Pass
TC-04	Delete operation	Remove document	Document is permanently deleted	Pass

9. OUTPUT

student_management_output_1.png.png

```
Terminal — node student_management.js

PS C:\Users\vinay\OneDrive\Desktop\full-stack-web-development\assignments\assignment2> node student_management.js

Connected to MongoDB successfully.
Students inserted successfully.

--- All Students ---
[
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', year: 3, marks: 85, email: 'ravi@example.com' },
  { rollNo: '23CM002', name: 'Priya Sharma', branch: 'CSE', year: 3, marks: 92, email: 'priya@example.com' },
  { rollNo: '23CM003', name: 'Ananya Verma', branch: 'IT', year: 2, marks: 78, email: 'ananya@example.com' },
  { rollNo: '23CM004', name: 'Kiran Patel', branch: 'ECE', year: 4, marks: 45, email: 'kiran@example.com' },
  { rollNo: '23CM005', name: 'Suresh Reddy', branch: 'CSE-AIML', year: 3, marks: 68, email: 'suresh@example.com' },
  { rollNo: '23CM006', name: 'Sneha Nair', branch: 'CSE', year: 2, marks: 88, email: 'sneha@example.com' },
  { rollNo: '23CM007', name: 'Ramesh Rao', branch: 'CSE', year: 3, marks: 95, email: 'ramesh@example.com' }
]

--- CSE-AIML Branch Students ---
[
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', year: 3, marks: 85 },
  { rollNo: '23CM005', name: 'Suresh Reddy', branch: 'CSE-AIML', year: 3, marks: 68 }
]

--- Students with Marks > 75 ---
[
  { rollNo: '23CM001', marks: 85 }, { rollNo: '23CM002', marks: 92 },
  { rollNo: '23CM006', marks: 88 }, { rollNo: '23CM007', marks: 95 }
]

--- Search Student by rollNo (23CM001) ---
{ rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', year: 3, marks: 85 }

Updated marks for 23CM001 to 92
Updated email for 23CM003
```

student_management_output_2.png.png

```
Terminal — node student_management.js (continued)

PS C:\Users\vinay\OneDrive\Desktop\full-stack-web-development\assignments\assignment2> node student_management.js

Deleted student record for 23CM005

--- Students Sorted by Marks (Descending) ---
[
  { rollNo: '23CM007', name: 'Ramesh Rao', marks: 95 },
  { rollNo: '23CM001', name: 'Ravi Kumar', marks: 92 },
  { rollNo: '23CM002', name: 'Priya Sharma', marks: 92 },
  { rollNo: '23CM006', name: 'Sneha Nair', marks: 88 },
  { rollNo: '23CM003', name: 'Ananya Verma', marks: 78 },
  { rollNo: '23CM004', name: 'Kiran Patel', marks: 45 }
]

Created unique index on rollNo

Index Search Stage: IXSCAN

--- Real-Time Extension Queries ---
Marks > 80: [ { rollNo: '23CM007', marks: 95 }, { rollNo: '23CM001', marks: 92 }, { rollNo: '23CM002', marks: 92 } ]
Marks < 50: [ { rollNo: '23CM004', marks: 45 } ]
Highest Scorer: [ { rollNo: '23CM007', name: 'Ramesh Rao', marks: 95 } ]
IT Branch: [ { rollNo: '23CM003', name: 'Ananya Verma', branch: 'IT', marks: 78 } ]
Sorted Ascending: [ { rollNo: '23CM004', marks: 45 }, { rollNo: '23CM003', marks: 78 }, { rollNo: '23CM006', marks: 88 } ]

Connection closed.
```

failed_test_output.png.png

```
Terminal — node test_negative.js

PS C:\Users\vinay\OneDrive\Desktop\full-stack-web-development\assignments\assignment2> node test_negative.js

Testing Duplicate Key on Unique Index 'rollNo_1':
MongoServerError: E11000 duplicate key error collection: collegeDB.students index: rollNo_1 dup key: { rollNo: 101 }
    at Collection.insertOne (node_modules/mongodb/lib/operations/insert.js:52:16)
    at run (C:\Users\vinay\OneDrive\Desktop\full-stack-web-development\assignments\assignment2\test_negative.js:14:20)

Searching non-existent rollNo '23CM999':
Query result: null
```

10. RESULT

Thus, the Student Database Management System was successfully implemented using MongoDB and Node.js. All CRUD operations, filtering, sorting, and indexing queries were executed and verified against expected outputs.